



Fig. 19. Large-scale search results and Billion-scale data.

19(b) shows that Starling is more than $2\times$ faster than DiskANN in the high recall regime (e.g., $Recall > 0.96$). Note that current vector databases use some data segmentation strategies and a query coordinator, which can avoid scanning all segments for a query [29]. Our method can work well with these strategies. For more details on data segmentation strategies, please refer to [23, 75].

7 DISCUSSION

Memory-based related work. Memory-based HVSS often involves preprocessing data to strike a balance between efficiency and accuracy. Existing algorithms fall into four categories: tree-based [20, 47, 50]; quantization- [9, 30, 36]; hashing- [27, 33, 43]; and graph-based [24, 25, 49]. Tree-based and hashing-based methods are not prevalent in HVSS due to the “curse of dimensionality” and low accuracy [45]. Quantization-based methods (e.g., IVFPQ) prove efficient and memory-saving but tend to suffer from a poor recall rate [35, 60]. Graph-based methods (e.g., HNSW) exhibit leading efficiency-accuracy trade-offs. However, they necessitate both raw vector data and the graph index to be in the main memory, elevating memory consumption and impeding scalability for large-scale vectors [16]. Starling is a universal and I/O-efficient framework capable of integrating different graph algorithms (e.g., HNSW) into the disk-index component. For example, Starling accommodates HNSW seamlessly without sacrificing functionality (cf. Fig. 16(b)).

Comparison analysis with SPANN. While both Starling and SPANN leverage in-memory graphs and locality-centric disk indexes, they address varying challenges and utilize unique strategies. SPANN, a clustering-based disk index, partitions data utilizing k-means, thereby achieving an inherent locality allowing clustered data to be stored and accessed synchronously. Furthermore, it builds an in-memory graph for fast cluster retrieval. In contrast, Starling addresses disk-based graph search by optimizing data layout and search strategy. When residing in memory, graph-based methods excel in terms of accuracy and efficiency. However, once placed on disk, they incur many I/Os due to long search path and poor data locality. Starling mitigates these issues by building an in-memory graph to shorten the search path and using block shuffling to enhance locality. Note that the graph index neighborhood exhibits both similarity and navigation traits [68]. This implies that a vertex may have neighbors from different clusters [49], posing a major challenge for the locality of the graph index. We compare block shuffling with a naive strategy that assigns vertices to blocks by k-means on SSNPP. The results show that block shuffling achieves a $12\times$ higher overlap ratio.

In-memory graph. Our in-memory graph serves as an index directing query routes, instead of acting as a cache for frequently accessed data. We initially used memory mapping (mmap) instead of direct I/O (`o_direct`) but found more disk I/Os linked to a memory-mapped graph. This lowers efficiency compared to the hot points strategy in the baseline framework. Our evaluation showed that the in-memory graph outperforms the hot points strategy in search performance and memory overhead. In Starling, we have the flexibility to utilize any graph algorithm like HNSW [49] or NSG [25] for the in-memory graph. We typically use the same algorithm for both in-memory and disk-based graphs. In HNSW, the upper-layer graphs are a subset of the layer-0 graph. Thus, we can keep the higher layers in memory as a multi-layered in-memory graph and store the layer-0 graph on disk. This makes HNSW easy to implement in Starling (see Fig. 16(b)).

Central assumption. With modern SSD (high IOPS), recent methods like DiskANN can fetch multiple blocks simultaneously in each disk round-trip. This is because fetching a small number of random blocks from a disk takes almost the same time as one block [35]. We keep this feature in Starling while increasing the vertex utilization ratio in each loaded block by enhancing locality. This reduces the total I/Os for a query by minimizing round-trips.

Data update. Starling, primarily emphasizing query optimization for static indexes, can handle incremental updates at the database level [29]. Some vector databases (e.g., ADBV [70]) segregate dynamic and static indexes to facilitate updates. The dynamic index, residing in memory, is built incrementally and uses a bitset to monitor deleted data. As incoming data continues to grow, the dynamic index expands correspondingly. Consequently, an asynchronous merging process transfers the burgeoning dynamic index to the disk-based static index, necessitating a comprehensive index reconstruction. Then, the block shuffling and in-memory graph techniques come into play.

Applications of range search (RS). RS is widely used in vector analytics, such as face recognition [62], near-duplicate detection [2], and various applications on general embeddings [58, 65, 69]. In some retrieval systems [14, 56], ANNS is followed by RS with seed vertices and similarity thresholds [31] to obtain a similar cluster and offer more comprehensive results [39]. For example, users can choose a seed vertex to get all related results. Several vector databases (e.g., PASE [72], VBase [76], Milvus [66]) and memory-based works [11, 46] support both ANNS and RS on the same dataset.

8 CONCLUSION

We conduct a study on HVSS for the data segment, which is an essential component in vector databases. HVSS for the data segment needs to meet strict requirements in terms of accuracy, efficiency, memory usage, and disk capacity. However, existing methods only address some of these aspects. Our framework, called Starling, adopts a disk-based graph index approach and considers all these requirements simultaneously. It optimizes the data layout and search strategy to minimize costly disk I/O operations. Experimental results demonstrate that Starling achieves significant performance improvements compared to state-of-the-art methods while maintaining a small space overhead. In the future, we plan to apply our methods to cache and GPU optimizations. We will also integrate Starling into Milvus for distributed optimization.

ACKNOWLEDGMENTS

This work was supported in part by the NSFC under Grants No. (62025206, U23A20296), the Primary R&D Plan of Zhejiang (2023C03198 and 2021C03156). Xiaomeng Yi and Xiaoliang Xu are the corresponding authors of the work.

REFERENCES

- [1] 2018. A Library for Efficient Similarity Search and Clustering of Dense Vectors. <https://github.com/facebookresearch/faiss>.
- [2] 2020. Using AI to detect COVID-19 misinformation and exploitative content. <https://ai.meta.com/blog/using-ai-to-detect-covid-19-misinformation-and-exploitative-content/>.
- [3] 2021. Billion-Scale Approximate Nearest Neighbor Search Challenge: NeurIPS’21 competition track. <https://big-ann-benchmarks.com/>.
- [4] 2021. Milvus Was Built for Massive-Scale (Think Trillion) Vector Similarity Search. <https://milvus.io/blog/Milvus-Was-Built-for-Massive-Scale-Think-Trillion-Vector-Similarity-Search.md>.
- [5] 2021. Scalable graph based indices for approximate nearest neighbor search. <https://github.com/microsoft/DiskANN>.
- [6] 2022. Building a Vector Database for Scalable Similarity Search. <https://milvus.io/blog/deep-dive-1-milvus-architecture-overview.md>.
- [7] 2023. The ChatGPT Retrieval Plugin lets you easily search and find personal or work documents by asking questions in everyday language. <https://github.com/openai/chatgpt-retrieval-plugin>.